

Toxic Comment Classification Project Report

1. Introduction

I chose the <Jigsaw Toxic Comment Classification dataset> because detecting harmful language online is a **real-world problem** that affects user safety on digital platforms. My goal was to build a model that predicts whether a comment is toxic or safe.

2. Dataset Description

The dataset originally has 8 columns, but I used 2: `comment\_text` as input and `toxic` as the target. I sampled 10,000 rows for this project. The dataset is imbalanced (90.58% Safe and 9.42% Toxic) so I used **F1-Score as my main metric**.

3. Models Used

I trained three models on the same data:

- ◆ Logistic Regression: *F1: 0.65*
- ◆ Random Forest: *F1: 0.50*
- ◆ Decision Tree: *F1: 0.49*

Logistic Regression was the best model. It achieved the highest *F1-Score* and the best balance between *Precision and Recall* for detecting toxic comments.

4. Neural Network

I built a neural network using Keras with one Dense layer (32 neurons, ReLU) and a Dropout layer (0.6) to reduce overfitting. The model was trained for 10 epochs.

The first version was overfitting-training loss dropped to near zero while validation loss kept rising. After increasing Dropout to 0.6 and reducing the layers, the model **became balanced** with training and validation loss close together.

5. Results & Comparison

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.93	0.61	0.70	0.65
Random Forest	0.92	0.59	0.43	0.50
Decision Tree	0.92	0.67	0.38	0.49
Neural Network	0.94	0.81	0.47	0.60

The **Neural Network** had *higher Precision (0.81)* but *lower Recall (0.47)*, giving an *F1-Score of 0.60*.

Logistic Regression remains the best model with **F1 of 0.65** and better **Recall** for catching toxic comments.

6. *What I Learned*

- Accuracy is misleading on imbalanced datasets, F1-Score is more reliable.
- Cleaning the text **before training** made a big difference in model performance.
- A simpler model like ***Logistic Regression*** can outperform a Neural Network on small text datasets.
- Overfitting can be reduced by **increasing Dropout** and **reducing model complexity**.